

RH Use Cases: Bluetooth

Objective

“SigCap collects Bluetooth MAC address, name, category, RSSI, and PHY type (Classic, LE 1M, LE 2M).

- We would love to have some sense of the frequency range of the Bluetooth signal
- I think it is not trivial to capture the exact frequency since it randomly hops frequency in a short interval.
- Not sure if there's any more information that we can get, but let me know if you find anything.”

Model for RadioHound: From RF signals get raw IQ internally, then BLE signal processing/detection inside RadioHound, lastly useful BLE measurements (Dr.Ghosh said that the phones chip does all the IQ data handling so the app only sees measurements, need to replicate this for Bluetooth)

During development, I will still do RadioHound IQ capture in Python in order to develop and validate the math, but raw IQ is not the desired final output.

NEED FREQUENCY BEHAVIOR: where in the 2.4 GHz spectrum Bluetooth activity is occurring despite Bluetooth rapidly hopping between frequencies

Investigation 1

BLE frequency hopping and its 40 RF channels,

Can an IQ recording identify when/where BLE energy or packets appear across those channels?

[Bg: IQ stands for Inphase (I) and Quadrature (Q) data. It represents a method of capturing and digitizing radio frequency (RF) signals by breaking them into two components that are 90 degrees out of phase with each other.]

frequency/channel ; detected BLE activity ; occupancy

BLE FREQUENCY HOPPING

40 radio channels spaced 2 MHz apart. 37, 38, and 39 are primary advertising channels used for discovery. The other 37 support connected communication and secondary advertising.

During a connection, the devices use a shared algorithm to select a channel at each connection event, so their frequency changes follow a coordinated pattern rather than independent random choices. A *channel map* identifies available channels and can be updated to avoid interference.

[Bluetooth SIG's Bluetooth LE Primer](#)

channel structure

[Silicon Labs' Adaptive Frequency Hopping guide](#)

channel selection and interference avoidance

BLUETOOTH LE CHANNEL LISTING		
CHANNEL FREQUENCY (MHZ)	CHANNEL NUMBER	CHANNEL APPLICATION
2402	37	P.Adv
2404	0	Data Ch
2406	1	Data Ch
2408	2	Data Ch
2410	3	Data Ch
2412	4	Data Ch
2414	5	Data Ch
2416	6	Data Ch
2418	7	Data Ch
2420	8	Data Ch
2422	9	Data Ch
2424	10	Data Ch
2426	38	P.Adv
2428	11	Data Ch
2430	12	Data Ch
2432	13	Data Ch
2434	14	Data Ch
2436	15	Data Ch
2438	16	Data Ch
2440	17	Data Ch
2442	18	Data Ch
2444	19	Data Ch
2446	20	Data Ch
2448	21	Data Ch
2450	22	Data Ch
2452	23	Data Ch
2454	24	Data Ch
2456	25	Data Ch
2458	26	Data Ch
2460	27	Data Ch
2462	28	Data Ch
2464	29	Data Ch
2466	30	Data Ch
2468	31	Data Ch
2470	32	Data Ch
2472	33	Data Ch
2474	34	Data Ch
2476	35	Data Ch
2478	36	Data Ch
2480	39	P.Adv

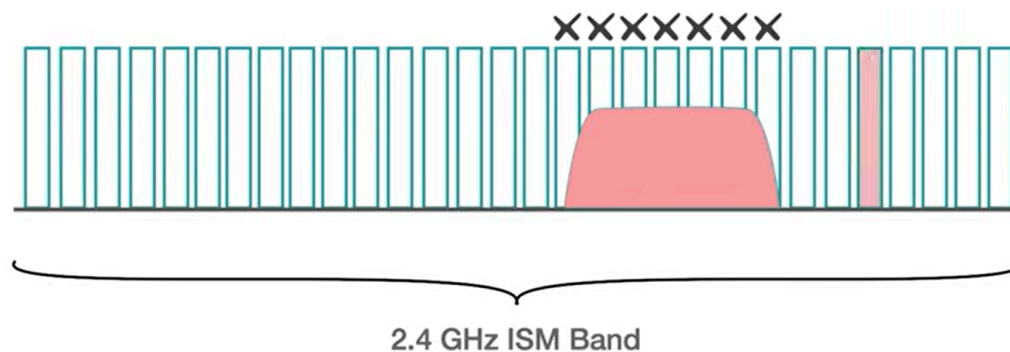
Figure 1.

Bluetooth Classic: uses 79, 1MHz channels hopping rate of 1600 channels per second

Bluetooth Low Energy: uses 40, 2MHz channels. 3 channels not used for hopping: advertising channels designated index 37, 38, and 39

Why do this? It's less than a millisecond per channel so if there is an interface only a small amount of data is lost, then it's easier to retransmit it later.

ADAPTIVE frequency hopping: Modern Bluetooth uses it so the system monitors the quality of channel for interface, if there is interference from wifi for example, it's tagged as noisy then it removes it and no longer hops to those channels. All without interruptions (realtime). The main goal is a really stable connection even without a lot of interference.



RadioHound Application:

Measure which BLE channels are active, rather than assigning each device one permanent frequency. Start with aggregate channel activity; following one particular connection is a separate, harder task because it requires connection information and timing. The BTLE receiver, for example, begins following hops after capturing connection-setup information. NOTE: narrow recordings cannot show activity on frequencies (verify RH bandwidth) outside its coverage.

Simulation Idea:

Begin with a small Python simulation that produces **connection event number ; time ; selected data channel**. MAIN IDEA: Implement a documented channel selection algorithm, then exclude several channels and check that the algorithm remaps selections to permitted channels

- [Zephyr's channel-selection implementation](#) algorithm
- [BTLE's Python baseband library](#) to add actual BLE packet waveforms later
- [PySDR's spectrogram example](#) to visualize waveforms across time/frequency

Algorithm #1: Channels are selected sequentially using fixed hop intervals passed while establishing connection.

Algorithm #2: The channel to be used is calculated from the current event counter and access address.

Investigation 2

Identifying BLE from IQ recordings

IQ recording preserves signal samples that software can analyze to find activity. A spectrogram divides those samples into short sections and calculates their frequency content, revealing when energy appears and which frequencies it occupies.

Identify BLE from other signals: the receiver must recover bits and check the expected packet structure. (CRC error checking). Background: PySDR's Frequency Domain chapter for IQ to spectrogram processing & [Bluetooth Core Specification's Link Layer chapter](#), (packet format and bit processing section)

RadioHound Application:

Recall Structure: RF > internal IQ > onboard BLE detection > Bluetooth measurements

Python processing used to develop and validate the math first. Save a recording with its center frequency/sample rate Aim to output channel, duration, packet count, and estimated occupancy. Calculate occupancy relative to the time that channel was actually observed, and keep unidentified energy separate from confirmed BLE packets.

Simulation Idea:

A good starting point is: [test_btler.py](#), generates advertising packet IQ, adds noise and frequency/sampling errors, and passes the result through a BLE receiver. Insert packets at known times, then use [PySDR's filtering/frequency shifting](#) to isolate channels or place simulated packets at different frequency offsets. Explore: [Rtone/sdr4iot-ble-rx](#) for packet detection and timestamp/frequency logging (not a sim)

Final Note: Given that I now have the SigCap app, I want to explore its data collection and use applications to get a better understanding.